

nhis-scrapper 운영 Runbook

문서 버전: v2 · 최종 수정일: 2026-07-08

수정 이력

버전	일자	주요 변경
v1	—	최초 작성
v2	2026-07-08	- 단위 테스트 규모 갱신 (59 suites / 554 tests) - NHIS 부하완화 환경변수에 진료내역 목록 동시성 2종 추가

항목	내용
제품명	nhis-scrapper
버전	0.1.0
공급자	렉스소프트
대상 독자	고객사 운영자 / 시스템 관리자
함께 보아야 할 문서	../../../../BUNDLE.md (배포 가이드 원문), ../구조/아키텍처.md (시스템 구성·환경변수), ../api문서/API.docx (호출자용 API 명세)

본 Runbook 은 다음 4가지 운영 시나리오를 다룹니다:

- 1. 배포 — 정상 배포·갱신 절차와 사전·사후 검증
- 2. 롤백 — 신규 번들이 실패했을 때 이전 버전으로 되돌리는 절차
- 3. 장애 대응 — 증상별 원인 식별·복구
- 4. NHIS 사이트 변경 대응 — NHIS 측 페이지·정책 변화에 따른 핫픽스 흐름

배포 환경 구축·.env 작성·install.sh 동작 원리는 ../../BUNDLE.md 가 단일 소스이며, 본 문서는 그 위에서 운영자가 반복적으로 수행하는 작업과 비상 시나리오에 초점을 둡니다.

1. 일상 운영 (빠른 참조)

실행 위치 규약

- ./install.sh 는 번들 루트(nhis-scrapper-bundle-<version>/)에서 실행
- 그 외 docker compose ... / curl .../health / .env 편집은 <bundle-root>/compose/ 에서 실행
- 본 문서의 모든 코드 블록 상단에 실행 위치를 # 실행 위치: ... 주석으로 명시

1.1 헬스체크

대상	방법
nginx 자체	curl -fsS http://localhost:\${NGINX_PORT:-80}/nginx-health → "ok"
WAS + Redis (readiness)	curl -fsS http://localhost:\${NGINX_PORT:-80}/health → 200 { status: "ok", redis: "ok" } . Redis 단절 시 503

LB / ALB 의 target health check 는 /health 사용을 권장합니다.

1.2 로그 위치 및 회전

로그	위치	회전
nginx access / error	compose/logs/nginx/{access,error}.log (호스트 bind mount)	매일 회전, LOG_RETENTION_DAYS 일 보관 후 자동 삭제
WAS / nginx stdout ·	docker compose logs <service> 또는 호스트 /var/lib/docker/containers/<id>/<id>-json.log	매일 회전, LOG_RETENTION_DAYS 일 보관 후 자동 삭제

로그	위치	회전
stderr		
Redis	고객사 ElastiCache 측 별도 관리	—

회전 설정 파일은 `/etc/logrotate.d/nhis-scrapper-nginx` 와 `/etc/logrotate.d/nhis-scrapper-docker` 두 개로, `install.sh` 가 자동 설치합니다. 보관 일수를 변경하려면 `.env` 의 `LOG_RETENTION_DAYS` 수정 후 `./install.sh` 재실행.

1.3 WAS 인스턴스 수 변경

```
# 실행 위치: <bundle-root>/compose
sed -i 's|^WAS_REPLICAS=.*|WAS_REPLICAS=8|' .env
docker compose up -d --scale was=8
```

1.4 재시작

이하 모두 `<bundle-root>/compose/` 에서 실행:

시나리오	명령
WAS 만 재시작 (코드 동일, 메모리 초기화)	<code>docker compose restart was</code>
nginx 만 재시작 (설정 변경 후)	<code>docker compose restart nginx</code>
전체 재시작	<code>docker compose restart</code>
환경변수(<code>.env</code>) 변경 후 적용	<code>docker compose up -d --force-recreate</code>

1.5 환경변수 변경

`.env` 수정 후 반드시 `docker compose up -d --force-recreate` 또는 `restart` 로 재기동해야 적용됩니다.
일부 변수는 컨테이너 부팅 시 Zod 스키마로 검증되므로 잘못된 값은 컨테이너가 즉시 실패합니다 — `docker compose logs was` 로 메시지 확인.

2. 배포

2.1 정상 배포 (초기 / 신규 번들 교체)

전체 절차의 표준 흐름은 `../BUNDLE.md` §배포 절차를 따릅니다. 본 절은 운영 관점의 체크포인트만 요약합니다.

사전 체크리스트

- ☐ 번들 무결성(SHA-256) 검증 — `MANIFEST.txt` 와 일치
- ☐ `.env` 의 필수값이 모두 채워짐 — `install.sh` 가 fail-fast 로 검증하지만 사전 확인 권장
 - `NODE_ENV` (`development` | `production`)
 - `AUTH_APP_KEY` , `AUTH_APP_SECRET`
 - `NHIS_SESSION_ENCRYPTION_KEY` (32자 이상)
 - `REDIS_HOST`
- ☐ nginx 노출 포트 `NGINX_PORT` 로의 인입이 보안그룹/방화벽에서 허용됨 (권장 토폴로지에서는 ALB → nginx 경로)

- ☐ nginx config 확인 - 고객사 환경에 맞추어 설정. (<bundle-root>/compose/nginx/nginx.conf), \$ 6 nginx 설정 커스터마이징 참조
- ☐ ElastiCache 엔드포인트 접근 가능 (보안그룹·VPC peering 등)
- ☐ 이전 버전 이미지 백업 확인 — § 3 롤백 대비

설치

```
# 실행 위치: <bundle-root>
./install.sh
```

사후 검증

```
# 실행 위치: <bundle-root>/compose
docker compose ps                # 모든 컨테이너 Up
curl -fsS http://localhost:${NGINX_PORT}/health    # 200 ok
docker compose logs --tail=100 --grep -iE "error|fatal" # 부팅 에러 없음
```

호출자(앱 서버) 측에서 /api/v2/nhis/start_auth 한 것을 시범 호출하여 sessionId 가 정상 반환되는지 확인합니다.

2.2 신규 번들 교체

```
# 실행 위치: <bundle-root>/compose

# 1. 현재 운영 중인 이미지 백업 (롤백 대비)
docker save nhis-scrapper:${docker compose config | grep 'image: nhis-scrapper' | awk -F: '{print $NF}')} \
-o ~/nhis-backups/nhis-scrapper-$(date +%Y%m%d).tar

# 2. 기존 컨테이너 정지
docker compose down --remove-orphans
```

```
# 실행 위치: <bundle-root>
./install.sh
```

.env 를 유지하려면 신규 번들의 compose/ 폴더로 기존 .env 를 복사 후 install.sh 실행.

3. 롤백

3.1 보존 정책

- 신규 배포 직전 docker save 로 현재 이미지의 tar 백업 보관 (위 § 2.2)
- compose/.env 의 직전 버전 사본을 .env.prev 등으로 보관

3.2 절차

```
# 실행 위치: <bundle-root>/compose

# 1. 현재 컨테이너 정지
docker compose down --remove-orphans

# 2. 이전 이미지 load
docker load -i ~/nhis-backups/nhis-scraper-<날짜>.tar

# 3. .env 의 APP_VERSION 을 백업된 버전으로 되돌림
sed -i 's|^APP_VERSION=.*|APP_VERSION=<이전 버전>|' .env

# 4. 기동
docker compose up -d
```

3.3 검증

롤백 직후 § 2.1 사후 검증 절차 반복.

4. 장애 대응

증상별 1차 원인·진단·복구 절차.
표의 모든 `docker compose ...` 명령은 `<bundle-root>/compose/` 에서 실행:

증상	1차 진단	복구
<code>/health</code> 가 503	<code>docker compose logs was</code> . Redis 연결 확인: <code>REDIS_HOST</code> / <code>REDIS_TLS</code> / 보안그룹 → <code>ElastiCache</code> 6379	<code>ElastiCache</code> 측 상태 확인 후 WAS 재시작 (<code>docker compose restart was</code>)
외부에서 <code>nginx</code> 응답 없음	호스트 방화벽·보안그룹의 <code>NGINX_PORT</code> 인입 허용 여부 확인	정책 갱신 후 재시도
<code>nginx</code> 가 502 / 504 반환	<code>docker compose logs was</code> — WAS 부팅 실패 (<code>env</code> 검증 / <code>ElastiCache</code> 단절 등). <code>docker compose ps</code> 로 WAS 컨테이너 상태 확인	WAS 환경변수·연결 정정 후 <code>docker compose up -d --force-recreate</code>
응답 지연 급증	<code>docker compose logs --tail=200 was</code> 로 <code>NHIS upstream</code> 응답 지연·재시도 로그. <code>NET_REQUEST_TIMEOUT_MS</code> 도달 여부	<code>NHIS</code> 측 장애일 수 있음 (§ 5 <code>NHIS</code> 사이트 변경 대응 참조). 일시적이면 자연 회복 대기
호출자 측에서 401 발생	<code>.env</code> 의 <code>AUTH_APP_KEY</code> / <code>AUTH_APP_SECRET</code> 가 호출자 헤더 값과 일치하는지	<code>secrets</code> 동기화 후 재시도
호출자 측에서 <code>NhisAuthSessionNotFoundError</code>	세션 TTL (<code>NHIS_SESSION_TTL_SECONDS</code> , 기본	TTL 조정 또는 호출자 측 흐름 검토

증상	1차 진단	복구
빈발	600s) 만료 또는 다른 인스턴스에서 이미 종료. 또는 ElastiCache 단절	
빈 응답 / 캡차 실패 폭증	NHIS 측 rate-limit / 차단 추정 — § 5.2 즉시 확인	§ 5.2 절차
WAS 만 한 대 떠 있음	.env 의 WAS_REPLICAS 미설정 또는 1	값 갱신 후 docker compose up -d --scale was=N
docker: permission denied	install.sh 후 셸 미재기동	재로그인 또는 newgrp docker

4.1 진단 명령 모음

```
# 실행 위치: <bundle-root>/compose

# 컨테이너 상태
docker compose ps

# 최근 로그 (실시간)
docker compose logs -f --tail=200 nginx was

# 특정 인스턴스 진단
docker exec -it $(docker compose ps -q was | head -1) sh

# Redis 도달성
docker exec -it $(docker compose ps -q was | head -1) sh -c 'echo "PING" | nc -w2 $REDIS_HOST $REDIS_PORT'
```

5. NHIS 사이트 변경·차단 대응

NHIS 는 외부 시스템이며 사전 통보 없이 페이지 구조·인증 정책·rate limit 등이 변경될 수 있습니다. 본 절은 그러한 변화를 감지·식별하기 위한 절차입니다.

5.1 페이지 구조 / 인증 정책 변경

증상

- 응답 본문은 200 인데 특정 데이터 항목이 누락되거나 비정상 형태로 들어옴
- 인증 단계에서 NhisAuthRequestFailedError / NhisLoginFailedError 가 빈번

식별 절차

- NHIS 데스크톱 / 모바일 페이지를 사람이 직접 접속해 변경 여부 확인 (UI / 폼 / 추가 본인확인 단계 등)
- 영향 받는 endpoint 식별 — docker compose logs was | grep -i "error" 로 오류 빈도·종류 집계
- 새 번들로 교체 시 § 2.2 신규 번들 교체 절차 적용

Step / Parser 변경 가이드 (개발자용 요약)

NHIS 페이지가 바뀐 경우 수정 위치:

변경 종류	위치
인증 흐름 단계 (handshake / authen-request / authen-result / login / 캡차 제출)	src/nhis/infrastructure/auth/steps/{start-auth,after-auth}/{web,mobile}/*.step.ts
데이터 페이지 HTML 파싱	src/nhis/infrastructure/parsers/{web,mobile}/<page>/*.parser.ts
호출 대상 URL · 공통 헤더 · 세션 타입 등 도메인 상수	src/nhis/domain/constants/
도메인 엔티티 (수집 데이터 shape)	src/nhis/domain/entities/

변경 절차:

- 1. 영향 받는 step / parser 파일 수정
- 2. 해당 *.spec.ts 의 픽처 (test/fixtures/) 를 새 NHIS 응답 샘플로 갱신
- 3. pnpm test 로 단위 테스트 통과 확인 (59 suites / 554 tests 기준)
- 4. bundle.sh full 로 새 번들 생성 후 § 2.2 신규 번들 교체 절차 적용

NHIS 가 새 인증 vendor 를 추가하거나 본인확인 단계가 늘어난 경우엔 step 트리에 새 단계를 끼우고 orchestrator(src/nhis/infrastructure/auth/{start,after}-auth.orchestrator.ts)의 호출 순서를 함께 수정해야 합니다.

5.2 NHIS 측 rate-limit / 빈 응답 차단

증상

- 본 제품은 정상 동작 (헬스체크 OK, env 정상)
- 그러나 NHIS 호출의 응답이 본문 빈 200 또는 특정 endpoint 만 일관되게 차단
- 단일 사용자 단위가 아니라 일정 시점 이후 다수 사용자에게 동시 발생

알려진 동작

- NHIS 모바일 채널 (m.nhis.or.kr) 의 경우, 같은 IP · 같은 세션 기준 짧은 시간 안에 동일 endpoint 호출 약 40회 이상에서 endpoint 별로 빈 응답이 반환되는 현상이 관측되어 있습니다.
- 정확한 임계값·차단 지속 시간·차단 단위(IP / 세션 / 사용자) 는 NHIS 측이 공개하지 않아 명확히 알려져 있지 않습니다.
- 데스크톱 채널 (www.nhis.or.kr) 도 정확한 rate limit 정책은 알려져 있지 않으며, 본 제품은 환경변수 조절을 통해 동시성을 제어합니다.

1차 대응 (운영자) — 즉각 완화

.env 에서 호출 부하를 낮추고 재시도 간격을 늘립니다 (변경 후 docker compose up -d --force-recreate):

환경변수	기본	조정 방향	의미
NHIS_MEDICAL_TREATMENT_DETAIL_CONCURRENCY	10	낮춤 (예: 3~5)	진료내역 상세 동시 호출 상한
NHIS_MEDICAL_TREATMENT_DETAIL_DELAY_MS	0	높임 (예: 200~500)	진료내역 상세 호출 간 최소 지연
NHIS_MEDICAL_TREATMENT_LIST_CONCURRENCY	10	낮춤 (예: 3~5)	진료내역 목록 페이지 병렬 조회 상한

환경변수	기본	조정 방향	의미
NHIS_MEDICAL_TREATMENT_LIST_DELAY_MS	0	높임 (예: 200~500)	진료내역 목록 호출 간 최소 지연
NHIS_NATIONAL_SCREENING_DETAIL_CONCURRENCY	5	낮춤 (예: 2~3)	건강검진 상세 동시 호출 상한
NHIS_NATIONAL_SCREENING_DETAIL_DELAY_MS	0	높임 (예: 200~500)	건강검진 상세 호출 간 최소 지연
NHIS_MAX_RETRY_COUNT	1	0~1 유지	빈 200 응답 재시도. 무리하게 늘리면 차단 가중 위험
NHIS_RETRY_DELAY_MS	100	300~1000	빈 200 응답 시 재시도 간격

조정 후 NHIS 응답률·에러 빈도가 회복되는지 모니터링합니다.

항구 대응 — 출발 IP 다양화

호출량 자체가 NHIS 임계값을 넘는 운영 부하라면, 출발 IP 를 분산시키는 편이 안전합니다.

본 제품은 다음 두 가지 운영 모드를 모두 지원합니다

- 1. NAT Gateway 를 AZ / subnet 별로 분리해 EIP 다수 확보
 - WAS 를 여러 private subnet 에 분산 배치하고, 각 subnet 의 라우트가 자기 AZ NAT Gateway 로 향하도록 구성
- 2. 프록시 풀 경유
 - .env 의 NET_PROXY_URL 에 회전형 프록시 (rotating proxy) 또는 다수 IP 풀을 가진 프록시 엔드포인트 지정

본 제품 측은 정확한 NHIS 정책을 모르며, 위 완화책은 관측된 현상 기반의 운영 가이드입니다.

5.3 모니터링·재발 방지

- 호출자(앱 서버) 측에서 NhisAuthRequestFailedError / 빈 응답 / 파싱 실패의 일별 비율을 추적 권장
- 임계치 도달 시 알람 → § 5.1 / § 5.2 절차 점검
- 주기적으로 담당자가 API 호출 결과 테스트 및 데이터 누락 확인 - 간편인증이 필요하여 테스트 자동화에 다소 한계 존재

6. nginx 설정 커스터마이징

본 제품의 deploy/nginx/nginx.conf 는 단일 server 블록 / server_name _; (모든 Host 헤더 수용) 으로 기본 배포됩니다.

배포된 설정파일 경로는 <bundle-root>/compose/nginx/nginx.conf 입니다.

고객사 운영 환경에 따라 server_name 조정이 필요할 수 있으므로 케이스별 가이드를 정리합니다.

수정 후 적용: docker compose restart nginx (실행 위치: <bundle-root>/compose/).

6.1 기본값

```
server {
    listen 80;
    server_name _;
    ...
}
```


- `server_name _;` → 모든 Host 헤더 수용. 단일 server 블록이라 자동으로 default server.
- ALB target health check 가 target IP 로 호출되도 (Host=IP) 그대로 통과.

6.2 케이스 A — 특정 도메인만 받기 (Host 헤더 strict 매칭)

Host 헤더 injection 차단을 명시적으로 적용하고 싶을 때 적용합니다.

nginx 의 `server_name` 매칭은 "미매치 = default server 로 fallback" 이므로, 단순히 `server_name api.example.com;` 만 적은 단일 server 블록은 strict 효과가 없습니다. 두 개의 server 블록으로 분리하여 default 를 444 로 닫아야 합니다.

```
# (1) default server - healthcheck 만 처리, 그 외는 차단
server {
    listen 80 default_server;
    server_name _;

    location = /nginx-health { access_log off; return 200 "ok\n"; }
    location = /health      { proxy_pass http://was:3000/health; }

    location / { return 444; } # 매치되지 않는 Host 는 연결 종료
}

# (2) 비즈니스 server - 정확한 도메인만 처리
server {
    listen 80;
    server_name api.example.com; # 또는 여러 도메인을 공백 구분
    ...                       # 기존 location / 블록 그대로 이동
}
```

주의: ALB target health check 는 target IP 로 호출되므로 (Host=IP), healthcheck 경로 (`/nginx-health` , `/health`) 는 반드시 **default_server** 블록에 두어야 합니다. 그렇지 않으면 strict 도메인 매칭에 걸려 모든 target unhealthy 가 됩니다.

6.3 케이스 C — nginx 단에서 TLS 종단

ALB 없이 nginx 가 직접 TLS 종단을 하는 경우 인증서 설정이 필요합니다.

```
server {
    listen 443 ssl;
    server_name api.example.com;
    ssl_certificate      /etc/nginx/certs/fullchain.pem;
    ssl_certificate_key  /etc/nginx/certs/privkey.pem;
    ...
}
```

- 인증서 파일을 `docker-compose.yml` 의 nginx 서비스 volume 으로 mount.
- 인증서 만료·교체 절차는 본 가이드 스코프 외 — 별도 검토 권장.

6.4 적용 확인

설정 변경 후 다음을 확인합니다 (실행 위치: `<bundle-root>/compose/`):

```
docker compose exec nginx nginx -t          # 문법 검증
docker compose restart nginx                # 재기동
docker compose logs --tail=50 nginx         # 부팅 에러 확인
curl -fsS http://localhost:${NGINX_PORT}/nginx-health # 200 ok
curl -fsS http://localhost:${NGINX_PORT}/health    # 200 ok (WAS readiness)
```

ALB 사용 환경에서는 ALB 콘솔에서 target health 상태도 함께 확인.
